

SMART INDIA HACKATHON 2026

TECHNICAL REPORT

AURYA

Confidence-Gated Virtual Camera Tracking System for FSOC Coarse Alignment

Problem Statement ID: SIH26169

Department of Space / ISRO

Theme: Smart Automation / Space Technology

Category: Software

Team Name: AURYA

Team ID: 129418

Team Members: Parth Raveriya, Kaushal Patel, Ravindra Arethiya ,

Nitin Gami, Hiya Modi, Manav Surti

Institution: Shri S'ad Vidya Mandal Institute Of Technology

Abstract

Free Space Optical Communication (FSOC) provides high-bandwidth wireless communication using optical signals instead of conventional radio-frequency links. However, the narrow beam divergence of optical communication systems makes accurate pointing, acquisition and tracking (PAT) essential, particularly when the communicating terminals are mobile. The coarse-alignment stage must initially locate the remote optical terminal or beacon within the camera field of view and continuously maintain its position despite target motion, camera motion, noise and atmospheric disturbances.

Aurya is a software-based virtual camera tracking system developed to address the coarse-alignment stage of mobile FSOC terminals. The system creates a configurable virtual environment containing a moving optical beacon and a movable virtual pan-tilt camera. The camera feed can be affected by different disturbances such as Gaussian, Poisson and salt-and-pepper noise, atmospheric conditions and motion disturbances. A computer-vision pipeline detects the beacon, while a Kalman filter estimates and predicts its position. A confidence-gated confirmation mechanism prevents premature locking onto noise, and a PID controller continuously adjusts the virtual camera to keep the beacon near the image center.

A major architectural feature of Aurya is its dual-input design. The same detection, tracking and performance-monitoring pipeline can process both the internal virtual simulator and externally supplied .mp4 video. This directly supports the Benchmark-2 evaluation requirement in which the supplied video bypasses the virtual PTZ camera.

The implemented prototype includes a graphical user interface, configurable simulation parameters, real-time tracking visualization and automatic performance logging. Testing was performed using multiple target-motion patterns, noise and atmospheric combinations, and two video resolutions. Reported test runs demonstrated average tracking errors within the 10-pixel target specification in the tested scenarios, while measured processing rates were substantially above the 20 FPS requirement. Some specification areas, including the complete 5–20 pixel beacon-size range, reacquisition performance and certain disturbance combinations, remain areas for further validation and improvement.

Keywords: Free Space Optical Communication, FSOC, Pointing Acquisition and Tracking, Virtual Camera, Optical Beacon, Computer Vision, Kalman Filter, PID Controller, Object Tracking, Coarse Alignment, Confidence-Gated Detection, Virtual PTZ, Image Processing.

Introduction and Problem Understanding

1. Background

Free Space Optical Communication is an optical wireless communication technology in which information is transmitted through a free-space optical beam. It can provide high data rates, operates in an unlicensed optical spectrum and is not affected by electromagnetic interference in the same manner as conventional RF communication.

The major challenge is the extremely narrow divergence of an optical communication beam. Even a small angular misalignment between two communicating terminals can cause the receiver to move outside the transmitted beam. Therefore, a communication terminal must perform Pointing, Acquisition and Tracking (PAT).

1. Pointing – directing the terminal toward the approximate location of the remote terminal.
2. Acquisition – detecting and identifying the remote optical beacon.
3. Tracking – continuously maintaining alignment as the target or platform moves.

2. Problem Statement

The fundamental problem addressed by Aurya is:

How can a virtual camera autonomously acquire, identify and continuously track a moving optical beacon despite noise, atmospheric disturbances and camera/platform motion while maintaining the target within a specified tracking-error limit?

Requirement	Official/Default Value
Virtual screen	At least 2000×2000 pixels
Camera resolution	640×480 pixels
FOV	$4^\circ \times 3^\circ$
Camera update rate	At least 30 Hz
Beacon size	5–20 pixels
Target motion	At least four motion patterns
Pan/tilt speed	5–10°/s
Acquisition time	≤ 2 s
Tracking error	≤ 10 pixels
Target loss	$< 5\%$
Processing speed	≥ 20 FPS

3. Why a Virtual System?

A conventional physical implementation would require camera hardware, a pan-tilt mechanism, an optical beacon, control electronics, controlled disturbance-generation equipment, and measurement and logging equipment. Such a setup increases cost, complexity and testing time.

Aurya instead creates a software-in-the-loop environment in which the camera, target and environment are simulated while the actual detection, estimation and control algorithms operate on generated image frames.

Objectives and Scope

1. Objectives

1. Develop a configurable virtual environment for FSOC coarse alignment.
2. Generate one or more moving optical beacon targets.
3. Simulate a movable virtual pan-tilt camera.
4. Automatically detect the optical beacon using computer vision.
5. Estimate and predict target position using a Kalman filter.
6. Prevent false lock caused by noise using confidence-gated confirmation.
7. Continuously control the virtual camera using closed-loop PID control.
8. Simulate realistic noise and atmospheric disturbances.
9. Provide real-time visualization of camera feed and tracking status.
10. Automatically calculate and store performance metrics.
11. Support externally supplied .mp4 video for Benchmark-2 evaluation.

12. Provide a modular architecture that can be extended with more advanced AI/ML detectors.

2. Scope

Included in the current implementation:

- Virtual scene generation
- Moving beacon simulation
- Virtual PTZ camera
- Multiple target-motion patterns
- Gaussian, Poisson and salt-and-pepper noise
- Atmospheric presets
- Computer-vision beacon detection
- Kalman filtering
- Confidence-gated lock confirmation
- Prediction-based detection gating
- PID camera control
- Search/scan mechanism
- Real-time GUI
- Performance logging
- Simulator input
- .mp4 input
- Multi-resolution validation

Existing Approach and Proposed Solution

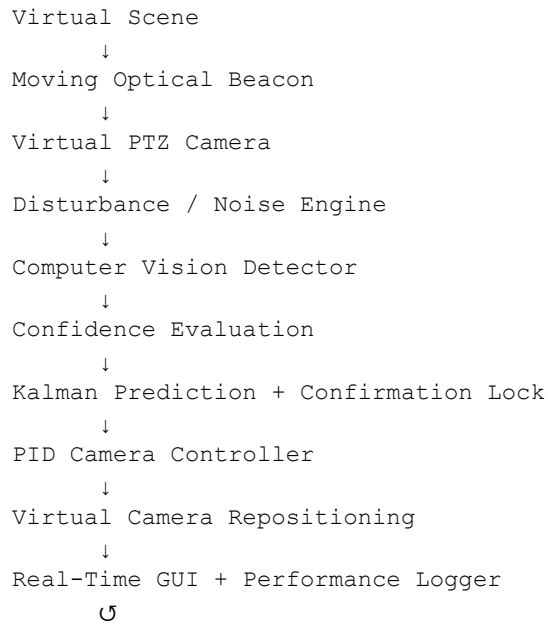
1. Conventional Approach

A straightforward tracking system could follow: Frame → Threshold → Find Bright Blob → Track Blob → Move Camera. Although simple, this approach can fail when noise creates a bright region that resembles the beacon.

- Salt-and-pepper noise may create isolated bright pixels.
- Atmospheric effects may reduce target contrast.
- Multiple bright objects may appear.
- Temporary detection failure can cause tracking instability.
- A false detection can permanently redirect the camera.

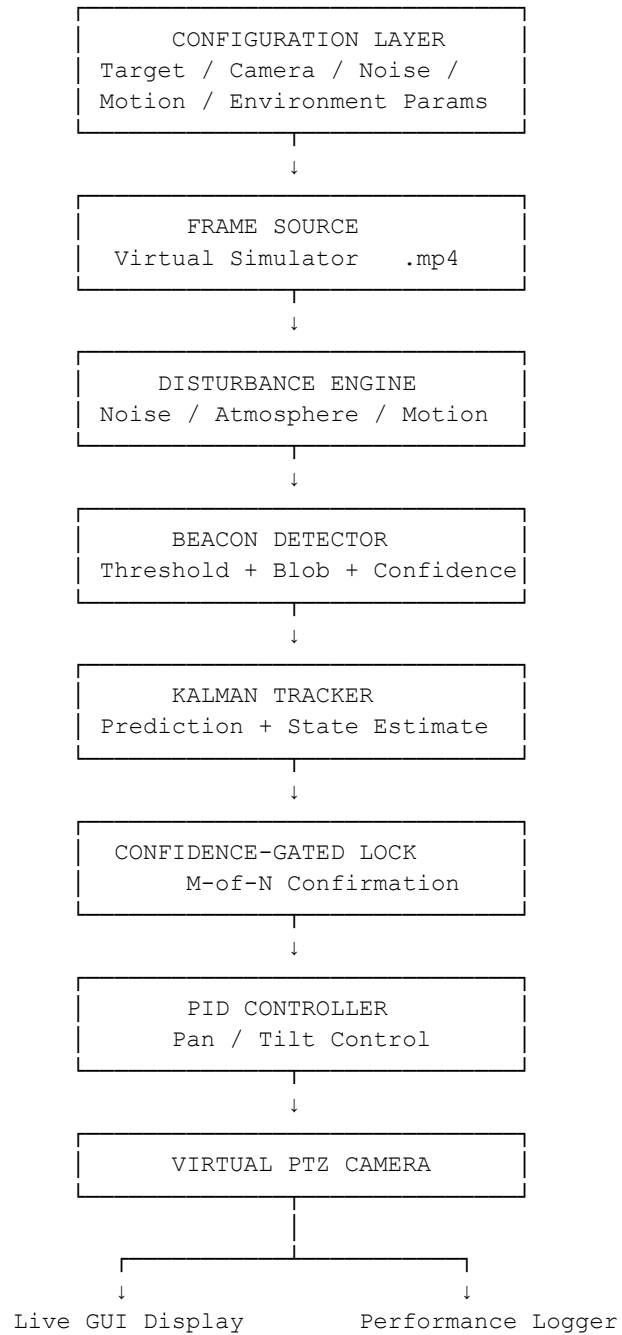
2. Proposed Approach

Aurya introduces additional decision and estimation layers: Detection → Confidence Evaluation → Confirmation → Kalman Prediction → Gated Tracking → PID Control.



System Architecture

1. Overall Architecture



Methodology and Algorithms

1. Scene Generation

The simulator uses a virtual environment of 2000×2000 pixels with a movable virtual camera. A beacon can follow straight-line, circular, figure-eight and random motion models. Additional spiral and sinusoidal motion patterns were considered in the simulator design. The simulator maintains target ground truth, allowing the estimated centroid to be compared against the actual target position.

2. Virtual Camera Model

Parameter	Value
Virtual scene	2000×2000 px
Camera resolution	640×480 px
Horizontal FOV	4°
Vertical FOV	3°
Camera update requirement	≥ 30 Hz
Implemented pan/tilt limit	$5^\circ/\text{s}$

The virtual camera acts as a crop/viewpoint inside the larger virtual environment. Instead of instantly teleporting the camera to the target, the controller produces movement commands, creating a closed-loop pointing simulation.

3. Beacon Detection

```
Input Frame
  ↓
Grayscale Conversion
  ↓
Median Blur (5×5)
  ↓
99.9th Percentile Threshold
  ↓
Connected Component / Blob Extraction
  ↓
Area Filtering
  ↓
Aspect Ratio Filtering
  ↓
Confidence Calculation
  ↓
Candidate Beacon
```

The implemented detector uses median filtering, percentile-based thresholding, connected-component analysis, area filtering, aspect-ratio filtering, and brightness/size-based confidence scoring.

4. Confidence Score

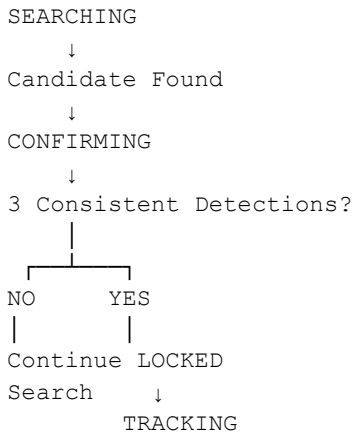
The detector combines brightness and target-size information:

$$C = 0.5 C_{\text{brightness}} + 0.5 C_{\text{size}}$$

A candidate is accepted only when its confidence exceeds the configured threshold.

5. Kalman Filter and Confirmation Lock

The tracker uses the state $X = [x, y, vx, vy]^T$ and follows a constant-velocity model. The filter predicts the next target position, compares new detections with the prediction, smooths noisy measurements, continues temporarily through missed detections, and rejects physically implausible detections.



The previous implementation required three consistent detections within approximately 25 pixels before confirming the target. After locking, detections sufficiently far from the Kalman prediction were rejected.

Camera Control

The horizontal and vertical image errors are calculated as:

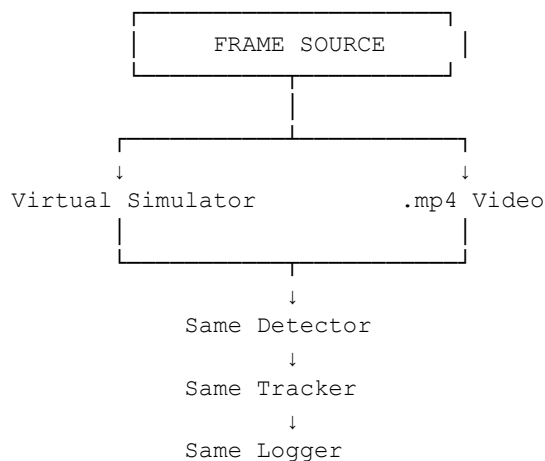
$$e_x = x_{\text{target}} - x_{\text{center}}$$

$$e_y = y_{\text{target}} - y_{\text{center}}$$

The PID controller is represented by $u(t) = K_p e(t) + K_i \int e(t)dt + K_d de(t)/dt$.

SEARCHING performs a raster scan, CONFIRMING freezes the camera while a candidate is verified, and TRACKING uses PID control to reduce the beacon's image error.

Dual-Input Architecture and Benchmark-2 Support



The common Frame Source abstraction ensures that the detector, tracker and performance-monitoring components do not depend on the origin of the frame. This is important because the official Benchmark-2 procedure provides .mp4 video at 30 FPS and requires the software to bypass its PTZ camera and use the video as input to the coarse-pointing system.

Technology Stack and Implementation

Layer	Technology
Programming Language	Python 3.13
Computer Vision	OpenCV
Numerical Processing	NumPy
Tracking	Custom Kalman Filter
Control	PID Controller
GUI	PyQt5
Real-Time Plotting	pyqtgraph
Data Processing	Pandas
Performance Logs	CSV / JSON
Packaging	PyInstaller
Input	Simulator + .mp4

The current implementation uses classical computer vision, estimation and control. Its intelligent behavior comes from confidence-based decision logic, Kalman state estimation, prediction-gated detection and closed-loop PID control.

User Interface and Performance Monitoring

- Camera feed display
- Beacon detection visualization
- Tracking centroid
- Camera center
- Tracking error
- Lock state
- Target state
- Configuration parameters
- Motion selection
- Disturbance configuration
- Real-time error graph
- Performance statistics

The logger records simulation duration, FPS, acquisition time, average tracking error, maximum tracking error, RMSE, lock retention, target loss, reacquisition information and processing time.

Testing and Validation

1. Testing Strategy

Testing was designed around reproducible scenarios. Seeded scenarios allow the same target trajectory and disturbance configuration to be repeated. Validation considered target motion, noise, atmospheric conditions, video input, resolution, tracking error, processing speed and lock retention.

2. Motion Scenarios

- Straight-line motion
- Circular motion
- Figure-eight motion
- Random motion

3. Disturbance Scenarios

- Gaussian noise
- Salt-and-pepper noise
- Poisson noise
- Fog
- Low-light
- Noise + fog stress video

Haze and rain were available as simulator conditions but were not sufficiently evidenced in the previous testing record to support measured-performance claims.

Performance Results

The following values are prototype test results recorded during the previous project-development testing. They should be treated as measured results for the tested scenarios, not as a claim that every official benchmark condition has been passed.

Test Scenario	Observed Result
Clean motion scenarios	2.2–4.8 px average error
Fog + Gaussian + Salt & Pepper	4.85 px average error
Low-light + Poisson	5.1 px average error
Noise + Fog stress video, 640×480	7.3 px average error
Video, 1280×960	4.0 px average error
Processing speed	65–270 FPS
Typical 640×480 processing	150+ FPS
Worst recorded FPS	~65 FPS at 1280×960
Post-lock loss in tested clean scenarios	0%
Fog/noise lock retention	99.3%

Acquisition Time

Acquisition time depended strongly on initial target location. Recorded tests showed approximately 0.07–0.17 seconds when the beacon began within the camera view, while a case with the beacon far outside the initial FOV took approximately 6.9 seconds.

Challenges and Solutions

Challenge	Implemented Solution
Bright noise can resemble beacon	Confidence-gated confirmation
False lock after one detection	Multiple consistent detections required
Temporary missed detection	Kalman prediction
Noisy centroid measurements	Kalman filtering
Target not initially visible	Raster search
Camera correction	PID control
Different input sources	Common Frame Source abstraction
Different video resolutions	Resolution-aware detector scaling
Repeatable testing	Seeded simulation
Evaluator evidence	Automatic performance logging

Limitations and Engineering Gaps

1. Atmospheric Coverage

Clear, Fog and Low-light conditions have recorded evidence. Haze and Rain are available as simulator conditions but were not sufficiently tested to support a measured-performance claim.

2. Camera Jitter and Platform Motion

The disturbance framework contains provisions for camera jitter and platform motion, but these were not comprehensively validated in the final testing batch.

3. AI/ML Limitation

The current system does not contain a trained neural-network detector. Its intelligent behavior comes from confidence-based decision logic, Kalman state estimation, prediction-gated detection and closed-loop PID control. A learned detector can be introduced later through the modular detector interface.

Innovation and Novelty

1. Confidence-Gated Acquisition – the system does not blindly lock onto the first candidate.
2. Prediction-Gated Tracking – Kalman prediction provides an expected target location for measurement validation.
3. Closed-Loop Virtual PTZ – detection is directly connected to camera control.
4. Disturbance-Aware Simulation – controlled noise and atmospheric degradation can be introduced.
5. Dual-Mode Architecture – the same processing pipeline operates on simulator and external .mp4 input.
6. Automatic Evaluation – performance metrics are recorded automatically.
7. Reproducible Testing – seeded scenarios allow identical conditions to be repeated.

Future Improvements

1. Improved Search Strategy

- Spiral search
- Coarse-to-fine search
- Probability-based search
- Motion-informed search
- Adaptive search based on previous target position

2. Re-acquisition Testing

```
LOCKED
↓
Beacon disappears
↓
Prediction / Coast
↓
Search
↓
Beacon returns
↓
Candidate detected
↓
Confirmation
↓
RE-LOCK
```

3. More Realistic Optical Simulation

- Beam divergence
- Optical blur
- Turbulence models
- Atmospheric attenuation
- Lens distortion
- Exposure variation
- Sensor saturation
- Dynamic background
- Realistic beacon intensity variation

4. Physical Hardware-in-the-Loop

The virtual camera could eventually be connected to a real camera, pan-tilt unit, embedded controller and optical beacon. The same software architecture could then transition from a fully virtual environment to hardware-in-the-loop testing.

Conclusion

Aurya was developed as a software-based virtual camera tracking system for the coarse alignment stage of mobile Free Space Optical Communication (FSOC) terminals. The system integrates a configurable virtual environment, moving optical beacon, virtual pan-tilt camera, disturbance simulation, computer-vision-based detection, Kalman state estimation, confidence-gated target acquisition, PID-based camera control, and automated performance monitoring into a unified tracking pipeline.

A key strength of the system is its confidence-gated tracking approach. Rather than relying solely on the brightest region in an image, Aurya requires multiple consistent observations before establishing a target lock. Once the target is acquired, Kalman prediction provides a reference for validating subsequent measurements, improving tracking stability in the presence of noise and temporary detection errors.

The system also provides a common frame-source architecture that supports both the internal virtual simulator and external .mp4 video input. This enables the same detection, tracking, control, and performance-monitoring pipeline to be used across development, testing, and video-based evaluation scenarios.

Prototype testing demonstrated average tracking errors below the specified 10-pixel requirement in the recorded test scenarios, while processing performance remained substantially above the required 20 FPS threshold. The evaluation also helped identify areas requiring further engineering validation, including complete testing across the specified beacon-size range, verification of reacquisition time, and comprehensive validation under all defined disturbance conditions.

Overall, Aurya provides a functional, modular, and extensible foundation for software-based FSOC coarse-pointing research. Its modular architecture allows future improvements such as lightweight learned detection, enhanced search and reacquisition strategies, more realistic optical and atmospheric disturbance models, and hardware-in-the-loop integration. These enhancements can further improve the system's robustness and support its progression toward a more comprehensive automated Pointing, Acquisition, and Tracking (PAT) platform.

References

1. Smart India Hackathon 2026, Problem Statement SIH26169, “Development of an AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile Free Space Optical Communication (FSOC) Terminals,” Department of Space / ISRO.
2. OpenCV Documentation – computer vision and image-processing functions.
3. NumPy Documentation – numerical computation and matrix operations.
4. PyQt5 Documentation – graphical user-interface development.
5. pyqtgraph Documentation – real-time plotting and visualization.
6. Pandas Documentation – data processing and performance-log analysis.
7. Python Documentation – Python programming language and standard libraries.
8. PyInstaller Documentation – Python application packaging and executable generation.
9. Development and testing records of the Aurya Virtual Camera Tracking System, including simulator, detector, tracker, controller, GUI and performance-validation experiments.